

설비 고장을 예측하는 AI 데이터 분석

고장 임박 분류 모델

RUL 기반 고장 임박 문제

C O N T E N T S

목차

01 RUL 기반 고장 임박 문제

02 라벨 생성 및 데이터 준비

03 랜덤 포레스트 고장 임박 분류

01

RUL 기반 고장 임박 문제

잔존 수명 RUL의 개념부터 분류 문제로의 재구성까지

우리가 만들 고장 임박 예측기

센서가 남긴 숫자에서 곧 멈출 설비를 미리 골라내는 작은 도구 만들기

01 · 무엇을 만드는가

센서 데이터로 곧 고장날 설비를 미리 골라내는 예측기 완성

02 · 어떻게 접근하는가

거창한 AI가 아니라 잘 만들어진 도구를 조립하는 방식

03 · 이번 파트의 약속

데이터로 더 나은 정비 판단을 내리는 단단한 첫 도구

완성된 예측기의 모습

데이터 불러오기부터 0.1 예측까지 한 줄기로 흐르는 네 단계

STEP 1

데이터 불러오기

NASA 항공기 엔진 데이터를 가공한 표 한 장을 컴퓨터로 읽어들이기

STEP 2

정답 라벨 만들기

각 설비가 곧 고장(1)인지 괜찮음(0)인지 표시하는 정답 컬럼 생성

STEP 3

패턴 학습

센서값과 정답의 관계를 컴퓨터가 스스로 익히도록 학습

STEP 4

예측 완성

새 데이터를 넣으면 0 또는 1로 답하는 예측기 완성

설비를 관리하는 세 가지 방식

사후보전·예방보전·예지보전 — 데이터로 갈수록 진화한 정비 패러다임

01 · 사후보전

고장 난 뒤 고치기

단순하지만 갑작스러운 정지와 큰 손실을
피할 수 없음

돌발 정지 위험



02 · 예방보전

정해진 주기에 점검

달력 기준으로 정기 점검 — 멀쩡한 부품까
지 미리 교체하는 낭비

교체 낭비



03 · 예지보전

데이터로 고장 시점 예측

설비의 실제 상태 데이터를 근거로 필요
한 때 정비

이번 파트의 주제

예지보전(PdM)이란 무엇인가

Predictive Maintenance — 달력이 아닌 설비의 실제 상태로 정비 시점 결정

정의

데이터로 고장 시점을 미리 내다보고 그 시점에 맞춰 정비

핵심

달력이 아니라 설비의 실제 상태를 근거로 판단

근거

온도·진동·압력 센서의 숫자 속에 고장 신호가 숨어 있음

잔존 수명(RUL)이란 무엇인가

Remaining Useful Life — 고장까지 앞으로 더 버틸 수 있는 양

RUL · REMAINING USEFUL LIFE

앞으로 더 버틸 수 있는 양

01 · RUL의 의미

RUL이 작아질수록 고장이 임박

RUL 50 → 앞으로 50만큼 더 작동한 뒤 고장

02 · RUL의 쓸모

정비 시점을 정하는 가장 직접적 근거

숫자 하나로 어디부터 점검할지 결정

RUL을 일상으로 이해하기

남을수록 RUL이 작아지는 친숙한 사례 세 가지

01 · 타이어

소모품의 잔존 수명

**새 타이어는 잔존 수명이 크고
남을수록 작아짐**

오래 탈수록 마모가 진행되어 교체 시점이 다가옴

교체 신호 = RUL이 거의 0

02 · 배터리

누적 사용에 따른 노화

**오래 쓸수록 빨리 닳아
교체 시점 근접**

앞으로 얼마나 더 쓸 수 있는지를 한 숫자로 표현

한 숫자로 요약된 미래

03 · 정비 우선순위

숫자 하나로 결정

**RUL이 작은 설비부터
먼저 점검**

모든 설비의 RUL을 알면 점검 우선순위를 쉽게 결정

분석가의 의사결정 근거

cycle로 RUL을 표현하는 방법

시간 단위는 시·분·초가 아닌 cycle — 설비가 한 번 작동한 운전 주기

$$\text{RUL} = \text{최종 고장 cycle} - \text{현재 cycle}$$

예시

200 cycle에 고장 나는 엔진이
현재 50 cycle이면 RUL은 150

$$200 - 50 = 150$$

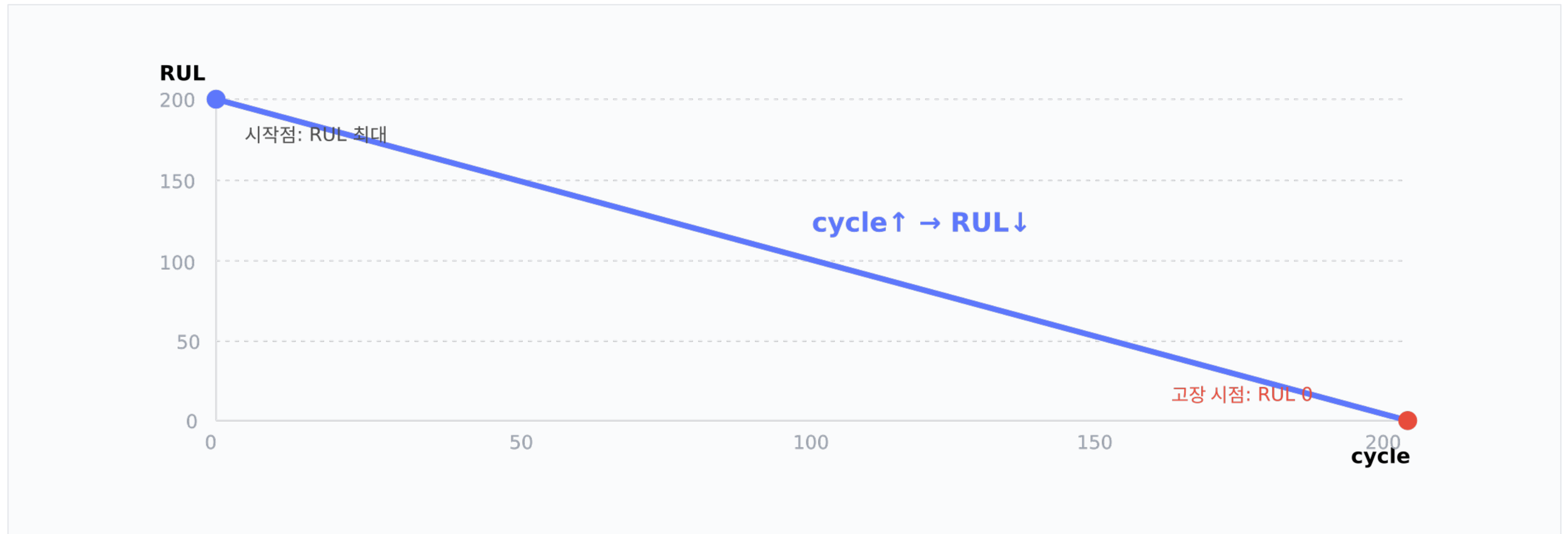
규칙

같은 설비라면
cycle↑ → RUL↓

데이터 전체를 관통하는 뼈대 규칙

cycle과 RUL의 관계 — 흐름 요약

cycle이 늘수록 RUL이 일직선으로 줄어드는 우하향 관계



cycle과 RUL은 시소 — 한쪽이 오르면 다른 쪽이 내려가는 관계

규칙 기반 방식의 한계

사람이 직접 임계값을 정하는 방식 — 변수가 많으면 사실상 불가능

Q1. 온도 90도를 넘으면 위험, 사람이 정한 규칙이면 충분한가

→ 고장은 온도 하나가 아니라 진동·압력·회전 속도 등 수십 개 값이 얹혀 발생

Q2. 88도는 안전하고 90도는 위험, 그 경계를 누가 정하는가

→ 정확한 기준을 사람이 알기 어려움 — 경험과 직관의 한계

Q3. 변수가 많아지면

→ 사람이 모든 규칙을 손으로 정하기는 사실상 불가능

머신러닝 — 데이터에서 규칙을 배운다

사람이 적어 둔 규칙 vs 데이터에서 스스로 찾아낸 패턴

01 · 규칙 기반

사람이 기준을 직접 정함

if 온도 > 90 then 위험 — 단순하고 이해하기 쉬움

한계

변수가 많아질수록 복잡한 관계에 약함

02 · 머신러닝

데이터에서 패턴을 스스로 찾음

정답 붙은 과거 데이터에서 고장으로 이어진 패턴 학습

강점

복잡한 관계에 강하고, 변수가 늘어나도 잘 작동

머신러닝이 '학습한다'는 말의 의미

사람이 정답 달린 예제를 풀며 감을 잡는 과정과 닮은 학습

STEP 1

예제 보여주기

입력과 정답이 짝지어진 사례를 많이 보여주기

STEP 2

오차 비교

자기 답과 실제 정답을 비교하며 기준을 조금씩 수정

STEP 3

반복 일반화

반복하면 처음 보는 데이터도 꽤 정확히 맞힘

STEP 4

학습 완료

이 과정 전체를 학습이라 부름

학습이 잘 되려면 정답이 붙은 사례가 충분히 많아야 함 — 그래서 좋은 데이터를 모으는 일이 중요

학습의 목표는 '일반화'

외운 문제만 풀 줄 알면 실력이 아님 — 처음 보는 데이터에서도 맞혀야 진짜 학습

01 · 무엇을 향해 학습하는가

학습은 외우기가 아니라 새 데이터에도 통하는 **일반화**가 목표

02 · 어떻게 실력을 가늠하는가

학습용과 **시험용 데이터**를 따로 떼어 시험용 성적으로 실력 측정

03 · 주의

사례가 적으면 **엉뚱한 규칙**을 만들기 쉬움

scikit-learn — 머신러닝 도구상자

머신러닝에 필요한 기능을 미리 만들어 둔 파이썬 라이브러리

scikit-learn

ML TOOLKIT FOR PYTHON

비유

요리할 때 칼과 냄비를 사와 쓰듯 부품을 조립

역할 분담

복잡한 계산은 라이브러리가 알아서 처리

우리가 할 일

무엇을 하려는지에만 집중

이번 파트에서 쓸 핵심 기능

지금은 이름만 가볍게 — 외우지 말고 흐름만 잡기

01 · 데이터 분리

train_test_split

데이터를 학습용과 평가용으로 나누기

02 · 분류 모델

**RandomForest
Classifier**

고장 임박을 분류하는 모델

03 · 학습

fit

모델을 학습시키기

04 · 예측

predict

새 데이터에 예측하기

각각 무슨 일을 하는지는 뒤에서 직접 써 보며 익힘 — 외우려 하지 말 것

C-MAPSS FD001 데이터셋 소개

NASA 항공기 엔진의 고장 과정을 시뮬레이션한 공개 데이터

01 · 출처

NASA가 항공기 엔진의 고장 과정을 시뮬레이션한 공개 데이터

02 · 규모

100대 엔진이 처음 상태에서 고장까지 간 전 과정 기록

03 · 우리 샘플

가장 단순한 FD001을 입문용으로 전처리한 샘플 사용

04 · 파일명

cmapss_fd001_sample.csv

데이터 컬럼 구조

컬럼은 식별·관찰·결과 세 묶음으로 나뉨

묶음	컬럼	역할
식별	unit_id, cycle	어느 설비의 몇 번째 시점인지 알려주는 인덱스
관찰	sensor_2 ~ sensor_15	설비 상태를 보여주는 센서값
결과	RUL, failure_soon	맞히고 싶은 값 — 잔존 수명과 고장 임박 라벨

컬럼의 세 가지 역할

뒤에서 x와 y를 나눌 때 결정적이 되는 컬럼 성격의 구분

01 · 식별 정보 데이터에 좌표 부여 unit_id cycle 어느 설비 어느 시점인지 알려줌 데이터의 주소 역할	02 · 관찰 값 모델이 볼 입력 후보 sensor_2 sensor_3 ... 설비의 상태를 보여줌 모델의 입력 x가 될 후보	03 · 결과 모델이 맞혀야 할 정답 RUL failure_soon 우리가 맞히고 싶은 값 모델의 정답 y가 될 후보
---	---	--

데이터 한 행이 담고 있는 정보

한 행 = 한 설비가 한 운전 시점에 보인 상태

$$\text{한 행} = \text{한 설비} \times \text{한 운전 시점}$$

예시

unit_id 1, cycle 50 행 = 1번 엔진의 50번째 시점 상태

한 설비는 시점마다 새 행 생성

같은 unit_id가 여러 행에 반복 등장

규모 감

200 cycle 산 엔진은 약 200줄을 차지

행을 따라 설비의 변화를 읽기

표를 가로로 vs 세로로 읽기 — 같은 데이터에서 두 가지 이야기

01 · 가로로 읽기

한 행 안의 여러 센서값

그 순간 설비의 **전반적 상태**

같은 순간에 측정된 여러 값이라 한 행이 곧 그 순간의 컨디션

02 · 세로로 읽기

같은 설비의 행들을 cycle 순서로

시간에 따른 **변화의 흐름**

센서값이 어떻게 변하고 RUL이 어떻게 줄어드는지 이야기처럼 보임

03 · 비유

해마다 모은 검진 기록

누적되면 **변화가 보임**

해마다 모은 건강검진 기록처럼 변화가 보이는 구조

cycle이 쌓일수록 RUL이 줄어든다

같은 설비 안에서 cycle이 1 늘면 RUL은 1 줄어드는 시소 관계

cycle **+1** → RUL **-1**

마라톤 비유

달린 거리 = cycle

남은 거리 = RUL

한 걸음 더 달릴수록 남은 거리는 줄어듦

핵심 규칙

cycle과 RUL은 시소처럼 반대로 움직임

이 우하향 관계가 데이터 전체의 뼈대

단, 서로 다른 설비끼리는 비교 불가

cycle↑ → RUL↓ 규칙은 같은 설비(같은 unit_id) 안에서만 성립

01 · 적용 범위

cycle↑ → RUL↓ 규칙은 같은 unit_id 안에서만 성립

02 · 함정

엔진마다 고장 시점이 달라 cycle만으로 비교 불가

03 · 예시

같은 50 cycle이라도 두 엔진의 RUL은 전혀 다를 수 있음

센서값이 고장을 미리 알려주는 원리

설비는 갑자기가 아니라 서서히 닳다가 고장에 이름

STEP 1

점진적 마모

설비는 갑자기가 아니라 서서히 닳다가 고장

STEP 2

센서값 변화

→ 닳아갈수록 온도가 오르거나 진동이 커짐

STEP 3

신호 해석

→ 이런 점진적 변화가 고장이 다가온다는 신호

STEP 4

전조 증상 비유

사람 몸의 미열·피로 같은 전조 증상과 닮음

하나의 센서만으론 잘 안 보여도 여러 센서를 함께 보면 점점 안 좋아진다는 그림이 또렷

여러 센서를 함께 읽는 일은 컴퓨터가 잘한다

사람의 한계가 컴퓨터의 강점으로 이어지는 지점

01 · 사람의 영역

소수의 변수

센서 두세 개 변화는 **눈으로 따라감**

그래프 두세 장 정도는 사람이 직접 비교하며
판단 가능

02 · 사람의 한계

다수의 변수

여섯·스물한 개가 **동시에 변하면 읽기
어려움**

상호작용까지 따라가기엔 인지 한계

03 · 머신러닝의 강점

동시 처리

여러 센서 조합을 한꺼번에 살펴 패턴
을 찾음

"이런 패턴일 때 고장 임박"이라는 관계를 데
이터에서 발견

우리 데이터에는 정답 라벨이 없다

RUL은 있지만, 곧 고장이냐는 예/아니오 정답은 우리가 직접 만들어야 함

01 · 가지고 있는 것

데이터에 RUL(고장까지 남은 cycle)은 있음

02 · 없는 것

하지만 곧 고장 vs 아니라는 예/아니오 정답은 없음

03 · 우리가 할 일

RUL을 근거로 그 정답을 직접 만드는 파생 라벨링

정답을 만드는 두 갈래 — 회귀와 분류

같은 데이터에서 정답을 정의하는 두 가지 방식

01 · 회귀(Regression)

RUL이라는 숫자 자체를 맞추기

"앞으로 47 cycle 남았습니다"처럼 구체적인 수치로 응답

출력 형태

연속 숫자 (RUL 47)

02 · 분류(Classification)

곧 고장인지 아닌지만 맞추기

"곧 고장(1) / 아직 괜찮음(0)"처럼 미리 정한 범주로 응답

출력 형태

0 또는 1 — 이번 파트의 선택

먼저 회귀를 살펴본 뒤 우리는 더 단순한 길, 분류를 선택

회귀 — 숫자(RUL)를 예측하기

연속된 숫자를 그대로 맞히는 방식 — 정밀하지만 매우 까다로움

정의

회귀는 RUL이 정확히 몇인지 **연속적인 숫자**를 맞히는 방식

장점

정보가 풍부해 정비 계획을 **정밀하게** 세울 수 있음

한계

숫자를 **정확히 맞히기**는 매우 까다로움 — 정답 경우의 수가 사실상 무한

분류 — 범주(곧 고장?)를 예측하기

정해진 범주 중 하나를 고르는 방식 — 우리가 택할 길

정의

분류는 정해진 범주 중 하나를 고르는 방식

우리의 범주

곧 고장(1)과 아직 괜찮음(0) 두 가지

현장 효용

정보는 덜 구체적이지만 현장 판단에 더 직관적

회귀와 분류 비교

출력.질문.난이도 — 세 축에서 본 두 방식의 차이

구분	회귀	분류
출력	연속 숫자 (RUL 47)	0 또는 1
질문	얼마나 남았나	곧 고장인가
난이도	상대적으로 높음	상대적으로 낮음

분류는 해석이 쉽다

결과를 행동으로 바로 옮길 수 있다는 점이 가장 큰 장점

01 · 행동 연결

1이면 점검 대상, 0이면 통과

모델 결과를 정비 행동으로 바로 매핑

당장은 정상 운전 / 우선 점검 — 둘 중 하나

02 · 평가 용이

맞았는지 확인이 간단

정답 경우의 수가 둘뿐이라 비교가 단순

예측 vs 실제 0/1을 한 칸씩 비교

입문에서 분류부터 시작하는 이유

쉽게 끝까지 완성해 보는 경험이 입문 단계에서 가장 중요

01 · 정답이 단순

0 아니면 1
무엇을 배울지 명확

정답 경우의 수가 둘로 좁혀져 학습 목표가 분명

02 · 해석이 직관적

1이면 점검
0이면 통과

결과를 현장의 정비 행동으로 바로 연결

03 · 확인이 쉬움

예측과 실제
0/1만 비교

한 칸씩 맞춰 보기만 하면 평가가 끝남

분류는 모든 머신러닝의 토대

분류로 큰 골격을 익히면 회귀로 확장하는 일도 훨씬 수월

01 · 전체 흐름 완성

분류로 전체 흐름을 한 번 완성하면 큰 틀이 손에 익음

02 · 확장 용이

그 토대 위에서 회귀처럼 어려운 방식으로 확장하기 쉬움

03 · 회귀의 자리

RUL을 직접 예측하는 회귀는 커리큘럼 마지막 단계에서 다룸

이진 분류와 0/1 라벨

정답이 두 가지 — 우리가 찾고 싶은 쪽을 1, 나머지를 0으로 표시

CLASS 1

1

곧 고장

관심 대상 · 찾아내고 싶은 경우

CLASS 0

아직 괜찮음

그 외의 나머지 경우

스팸 메일은 1, 정상 메일은 0으로 두는 것과 같은 약속

왜 글자 대신 0과 1을 쓰나

숫자 표시 하나로 계산·비교·집계가 한꺼번에 편리

01 · 계산 편리

컴퓨터는 숫자를 다루는 데 능함

모델 내부의 계산이 숫자 기반이라 자연스러운 표현

02 · 비교 편리

글자보다 숫자가 비교에 유리

예측 == 실제 같은 단순 비교 한 줄로 평가 가능

03 · 집계 편리

1의 개수를 세면 곧 고장 건수

합계만으로 "오늘 점검 대상이 몇 대?"를 파악

failure_soon 규칙 정하기

RUL이 기준값 이하면 1, 아니면 0 — 회귀를 분류로 다시 쓴 한 줄

FAILURE_SOON RULE

$\text{failure_soon} = 1 \text{ if } \text{RUL} \leq \text{기준값} \text{ else } 0$

예시 — 기준값 30

RUL 30 이하는 1, 31 이상은 0

문제의 재구성

어려운 회귀 문제를 쉬운 분류 문제로 다시 쓴 것

RUL에서 failure_soon으로

기준값 30 —연속적인 RUL이 두 값으로 접히는 모습

RUL	failure_soon
-----	--------------

150

80

30 · 경계(이하 포함)

12

failure_soon은 파생 라벨

원본에 없던 컬럼을 RUL로부터 만든 것 — 기준값 선택이 매우 중요

01 · 정의

원래 없던 값을 RUL로부터 만든 것 — 파생 라벨

02 · 핵심 선택

기준값을 몇으로 잡느냐가 매우 중요한 선택

03 · 다음 모듈 예고

이 기준값 이야기는 다음 모듈에서 깊이 다룸

분류 결과를 정비 판단으로 연결하기

분석가의 일은 모델을 돌리는 데서 끝나지 않음 — 현장이 쓸 판단으로 번역

01 · 안 좋은 끝맺음

모델의 0과 1이 **화면에만 머물면 쓸모 없음**

02 · 분석의 목적

결과를 실제 **정비 행동**으로 옮기는 일

03 · 분석가의 일

결과를 현장이 쓸 판단으로 번역

0과 1을 점검 여부로 읽기

모델은 최종 결정이 아닌 판단을 돕는 조수

예측 1

곧 고장

우선 점검 대상에 올리기

1로 예측된 설비들을 모아 우선순위 목록 구성 가능

예측 0

아직 괜찮음

당장은 정상 운전 이어가기

당장 정비 자원 투입 불필요

참고 자료

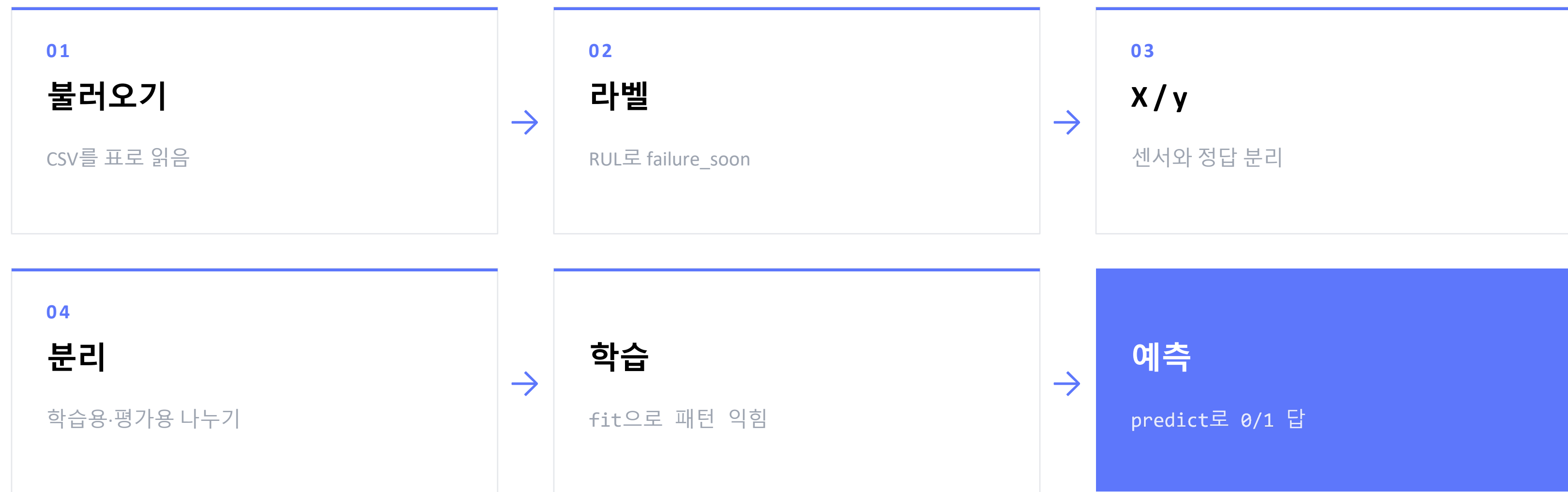
조수로 활용

모델은 최종 결정이 아닌 판단을 돕는 조수

예측이 항상 옳지는 않으니 참고 자료로 활용

전체 작업 흐름 — 6단계 파이프라인

물이 관을 따라 흐르듯 데이터가 단계를 거치며 예측 결과로 변해 감



이런 일련의 과정을 파이프라인이라 부름

각 단계가 하는 일 — 데이터 준비

1·2·3 단계 — 모델에게 보여 줄 자료를 정리

STEP 1

불러오기

read_csv로 CSV를 표로 읽기

pandas로 다룰 준비

STEP 2

라벨

RUL로 failure_soon 만들기

파생 라벨 생성

STEP 3

x/y 나누기

입력 센서들과 정답 분리

x = 보여 줄 입력, y = 맞힐 정답

각 단계가 하는 일 — 학습과 예측

4·5·6 단계 — 가르치고 시험 본 뒤 예측까지

STEP 4

분리

학습용과 평가용으로 나누기

가르칠 데이터와 시험 볼 데이터 구분

STEP 5

학습

fit으로 패턴 익히기

학습용 데이터를 모델에 보여 줌

STEP 6

예측

predict로 0/1 답 받기

여섯 단계가 끝나면 예측기 완성

지금 우리는 어디에 있나

전체 지도에서 오늘 위치를 알면 길을 잃지 않음

MODULE 1 · 현재 위치

1단계 — 불러오기

데이터를 불러와 충분히 이해하는 데 집중

이번 시간의 목표

MODULE 2 · 다음

라벨 · x/y · 분리

failure_soon 라벨 생성과 데이터 분리 작업

2·3·4 단계

MODULE 3 · 마지막

모델 학습 · 예측

랜덤 포레스트 분류기로 학습·예측까지 완성

5·6 단계

02

실습 — pandas로 C-MAPSS 데이터 탐색

데이터 불러오기부터 한 설비의 일생 따라가기까지 직접 손으로

실습 — 라이브러리 임포트와 첫 불러오기

내내 다룰 데이터를 컴퓨터로 불러와 첫인사를 나누는 단계

OBJECTIVE · 이번 실습의 목표

pandas를 불러오고 `cmapss_fd001_sample.csv`를 표로 읽어 첫 모습 확인

앞 단원에서 배운 pandas를 그대로 활용

DATASET · 사용 데이터

`unit_id`, `cycle`, `sensor × 6`, `RUL`

입문자가 다루기 좋게 정리된 샘플 — 잘 골라 둔 표를 믿고 분석에 집중

실습 — 불러오기 코드

pandas를 pd로 부르고 read_csv로 파일을 df라는 표에 담기

CODE · LOAD_CSV.PY

```
# pandas를 pd라는 별명으로 불러오기
import pandas as pd

# CSV 파일을 읽어 df라는 표에 담기
df = pd.read_csv('cmapss_fd001_sample.csv')
```

CHECKPOINT

파일을 못 찾으면 코드와 같은 폴더에 csv가 있는지 확인

실습 — 첫 모습 확인

head로 첫 5행을, shape로 행과 열 개수를 확인

CODE · HEAD_SHAPE.PY

```
# 첫 5행 확인
df.head()

# 행과 열의 개수 확인
df.shape
```

CHECKPOINT

unit_id · cycle · 센서들 · RUL이 보이는가, shape 결과가 (행, 열) 형태인가

실습 — 데이터 크기 점검

shape로 전체 행과 열의 개수를 다시 확인해 규모 감을 잡기

CODE · SHAPE.PY

```
# 데이터의 전체 크기 확인
df.shape
```

CHECKPOINT

데이터가 얼마나 큰지 감을 잡아 두기 — 다음 단계로 넘어가기 전 짚어 둘 숫자

실습 — 컬럼 이름 확인

분석 전 데이터가 어떻게 생겼는지 살펴보는 일은 길 떠나기 전 지도 확인

CODE · COLUMNS.PY

```
# 컬럼 이름 목록 확인  
df.columns
```

CHECKPOINT

어떤 센서들이 있는지 눈에 익혀 두기 — 이후 분석에서 자주 쓰임

실습 — 자료형과 결측 요약

info는 각 컬럼의 자료형과 값 개수를 요약 — 결측 단서까지 한눈에

CODE · INFO.PY

```
# 각 컬럼의 자료형과 값 개수 요약  
df.info()
```

CHECKPOINT

모든 컬럼의 값 개수가 동일한가 — 다르다면 결측치가 있다는 신호

실습 — 기초 통계 살펴보기

describe는 숫자 컬럼의 평균·최소·최대 같은 기초 통계 한눈에

CODE · DESCRIBE.PY

```
# 숫자 컬럼의 평균·최소·최대 같은 기초 통계 보기
df.describe()
```

CHECKPOINT

RUL의 최솟값·최댓값이 합리적 범위인지, 센서별 값 범위가 많이 다른지 확인

실습 — 마지막 행 확인

tail로 마지막 5행을 보고 데이터가 끝까지 잘 들어왔는지 확인

CODE · TAIL.PY

```
# 마지막 5행 확인  
df.tail()
```

CHECKPOINT

데이터가 끝까지 잘 들어왔는지 확인하는 습관 들이기

실습 — 설비 대수 세기

nunique는 서로 다른 값의 개수 — unit_id에 적용해 설비 대수 확인

CODE · NUNIQUE.PY

```
# 서로 다른 설비(unit_id)의 개수 = 설비 대수
df['unit_id'].nunique()
```

CHECKPOINT

결과로 나온 설비 대수를 말로 설명할 수 있어야 함

실습 — 결측치와 센서 범위 점검

빠진 값과 센서마다 값 크기 차이 — 분석 전 빈틈 점검부터

OBJECTIVE · 이번 실습의 목표

컬럼별 결측치 개수를 세고 센서값 범위를 비교

앞 단원의 결측치 확인 방법을 다시 활용

PRINCIPLE · 분석의 원칙

좋은 분석은 데이터의 빈틈을 먼저 점검하는 데서 시작

결측·이상치를 외면한 채 분석하면 결과 전체가 흔들림

실습 — 결측치 세기

isna와 sum을 이어 쓰면 컬럼별 결측치 개수 한눈에

CODE · ISNA_SUM.PY

```
# 컬럼별 결측치 개수 세기  
df.isna().sum()
```

CHECKPOINT

모든 컬럼에서 0이 나오는지 확인 — 0이면 결측치가 없다는 뜻

실습 — 한 센서 분포 보기

특정 센서 하나에 describe를 적용해 값의 분포를 자세히 보기

CODE · SENSOR_DIST.PY

```
# sensor_2 하나의 값 분포 확인  
df['sensor_2'].describe()
```

CHECKPOINT

개별 컬럼에 describe를 적용하면 그 컬럼만의 통계가 나옴

실습 — 여러 센서 평균 비교

센서 여러 개의 평균을 한 번에 비교

CODE · SENSOR_MEAN.PY

```
# 여러 센서의 평균을 한 번에 비교
df[['sensor_2', 'sensor_4', 'sensor_11']].mean()
```

CHECKPOINT

센서마다 평균 크기가 서로 크게 다른 것을 확인 — 모델 다룰 때 다시 나옴

실습 — 한 설비의 일생 따라가기

1번 엔진 하나만 떼어 내어 처음부터 고장까지 행을 따라 읽기

OBJECTIVE · 이번 실습의 목표

설비 한 대만 골라 **cycle 순서**로 RUL과 센서 변화 관찰

앞에서 배운 조건 필터링 그대로 활용

SCOPE · 분석 범위

전체가 아니라 1번 엔진 하나에 집중

개별 설비의 이야기를 읽고 나면 전체 데이터를 더 깊이 이해

실습 — 한 설비만 골라내기

unit_id가 1인 행만 골라 unit1에 담고 행 수 확인

CODE · PICK_UNIT.PY

```
# unit_id가 1인 행만 골라 unit1에 담기
unit1 = df[df['unit_id'] == 1]

# 이 설비가 몇 cycle을 살았는지 확인
unit1.shape
```

CHECKPOINT

shape로 본 행 수 = 이 엔진이 산 cycle 수

실습 — 시작과 끝의 RUL 비교

이 설비의 시작 부분과 끝부분 RUL을 한눈에 비교

CODE · UNIT1_RUL.PY

```
# 1번 엔진의 시작 부분 cycle·RUL
unit1[['cycle', 'RUL']].head()

# 1번 엔진의 끝부분 cycle·RUL
unit1[['cycle', 'RUL']].tail()
```

CHECKPOINT

head는 RUL이 크고 tail은 작아짐 — 마지막 행의 RUL이 0에 가까운 지점이 고장

실습 — RUL 변화 그래프 그리기

cycle에 따른 RUL 변화를 선 그래프로 그려 우하향 직선 확인

CODE · UNIT1_PLOT.PY

```
# matplotlib 불러오기
import matplotlib.pyplot as plt

# cycle에 따른 RUL 변화 그래프
plt.plot(unit1['cycle'], unit1['RUL'])
plt.xlabel('cycle')
plt.ylabel('RUL')
plt.show()
```

CHECKPOINT

그래프가 오른쪽 아래로 내려가는 우하향 직선인가 — cycle↑ → RUL↓

실습 — 두 설비의 수명 비교

설비끼리는 cycle만으로 비교 불가 — 두 설비를 나란히 두고 직접 확인

OBJECTIVE · 이번 실습의 목표

두 설비의 최대 cycle 비교 + 같은 cycle에서 상태 차이

cycle만으로 설비를 비교할 수 없다는 사실을 눈으로 확인하는 단계

KEY POINT · 확인 포인트

설비마다 고장까지 버티는 cycle 수가 다름을 직접 확인

어떤 엔진은 130 cycle에, 다른 엔진은 250 cycle에 고장 — 단순 비교 불가

실습 — 설비별 수명 길이 구하기

groupby로 설비별 최대 cycle(=수명 길이)을 구하고 정렬

CODE · UNIT_LIFE.PY

```
# 설비별 가장 큰 cycle = 수명 길이  
life = df.groupby('unit_id')['cycle'].max()  
  
# 오래 산 설비·일찍 고장난 설비 한눈에  
life.sort_values()
```

CHECKPOINT

설비마다 최대 cycle이 서로 다른 것을 확인했는가

실습 — 같은 cycle에서 RUL 비교

두 설비의 같은 50 cycle 시점을 각각 골라 RUL 비교

CODE · COMPARE_AT_50.PY

```
# 1번 엔진의 50 cycle 시점
df[(df['unit_id'] == 1) & (df['cycle'] == 50)]

# 2번 엔진의 50 cycle 시점
df[(df['unit_id'] == 2) & (df['cycle'] == 50)]
```

CHECKPOINT

같은 50 cycle인데 두 설비의 RUL이 다른가

실습 — 비교에서 얻는 교훈

설비마다 고장 시점이 다르므로 cycle만으로는 비교 불가

01 · 직접 확인한 사실

같은 cycle이라도 설비마다 RUL이 다름

02 · 분석상의 결론

cycle만으로는 설비끼리 직접 비교할 수 없음

03 · 모듈 1 마무리 · 다음으로

다음 모듈 — failure_soon 라벨 생성과 데이터 준비